ADVANCED DATABASE SYSTEMS PORTFOLIO 4

MIT 8103 - Advanced Database Systems

Student Name: Oluwatosin Ogunjemilua
Student ID: 301904697

Matriculation Number: 2026/A/MIT/0050
Academic Session: 2026/2027
Course: MIT 8103 Advanced Database Systems

Git Hub Repository url: https://github.com/TosinKyle/MIT8103-Advanced-Database-Systems-Apex-Digital-Bank

PORTFOLIO 4: NoSQL and Advanced Data Models

Objective

To demonstrate the design and implementation of NoSQL and advanced data models for the Apex Digital Bank case study while evaluating their suitability for modern banking applications.

Introduction

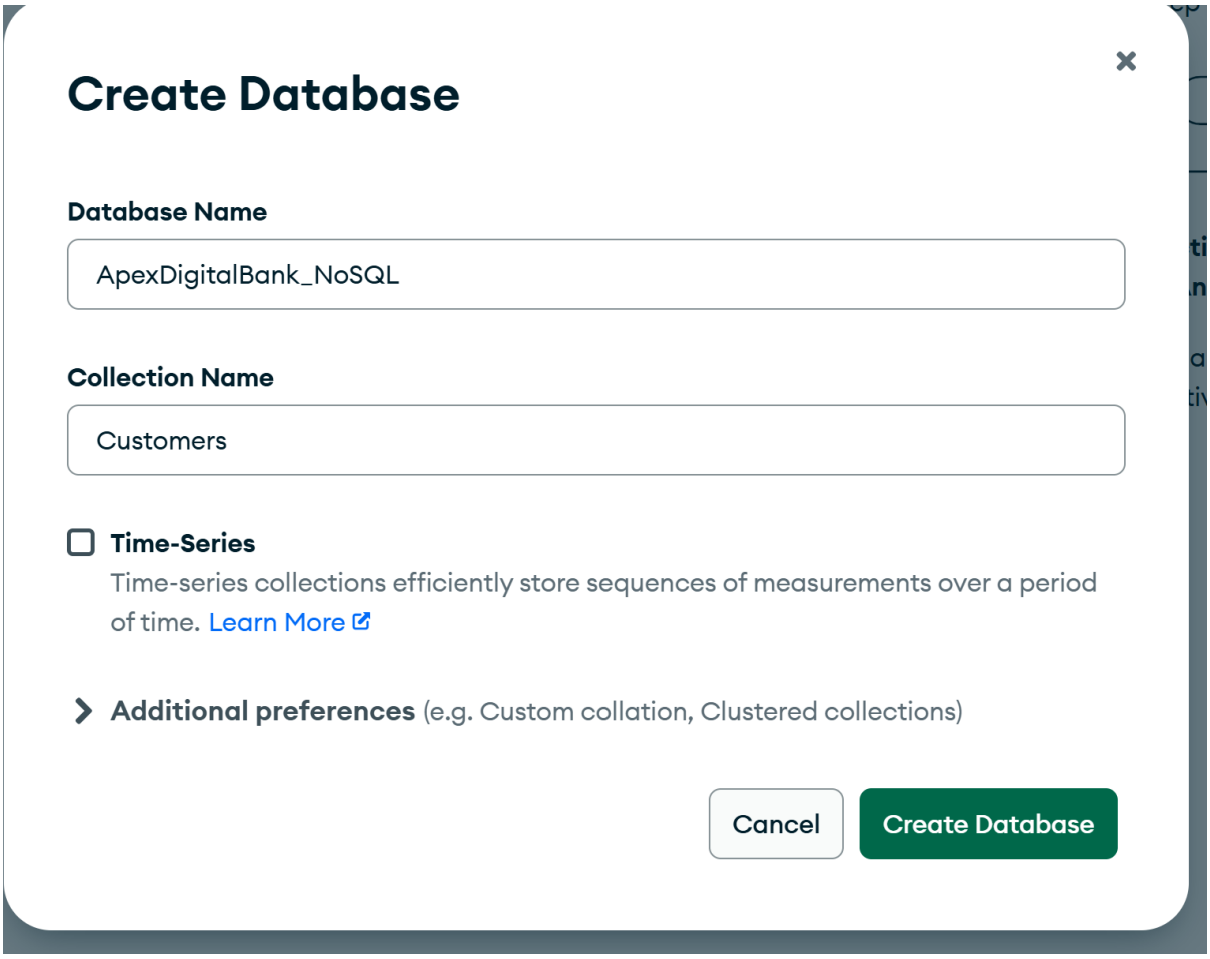
The Apex Digital Bank database was originally implemented using a relational database management system (SQL Server). To support scalability, flexibility, and high-volume data processing, the database was redesigned using NoSQL concepts and advanced data models. This portfolio demonstrates document, key-value, column-family, and graph database approaches using the same banking case study.

1. Document Model (MongoDB)

MongoDB is a popular open-source NoSQL database used to store, manage, and retrieve large amounts of data efficiently. Unlike traditional relational databases that store data in tables and rows, MongoDB stores data in flexible, JSON-like documents called BSON documents.

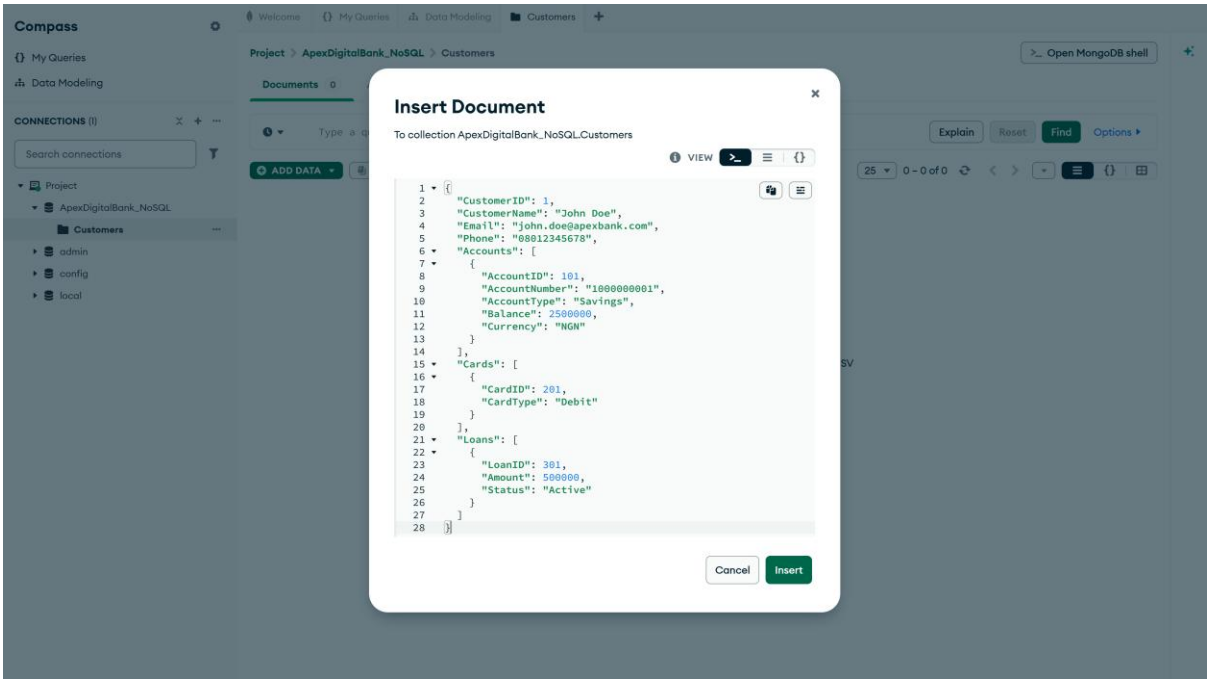
The document model stores data in JSON-like documents.

Create Database ApexDigitalBank_NoSQL and customer collection

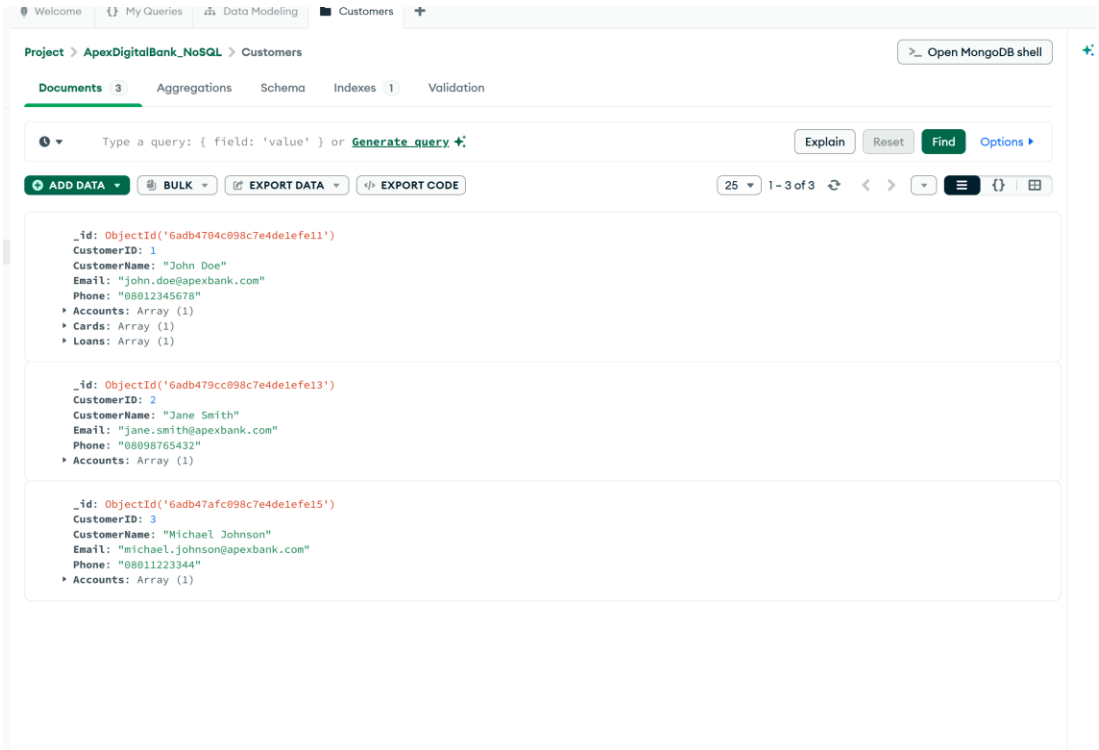


Create Customer documents

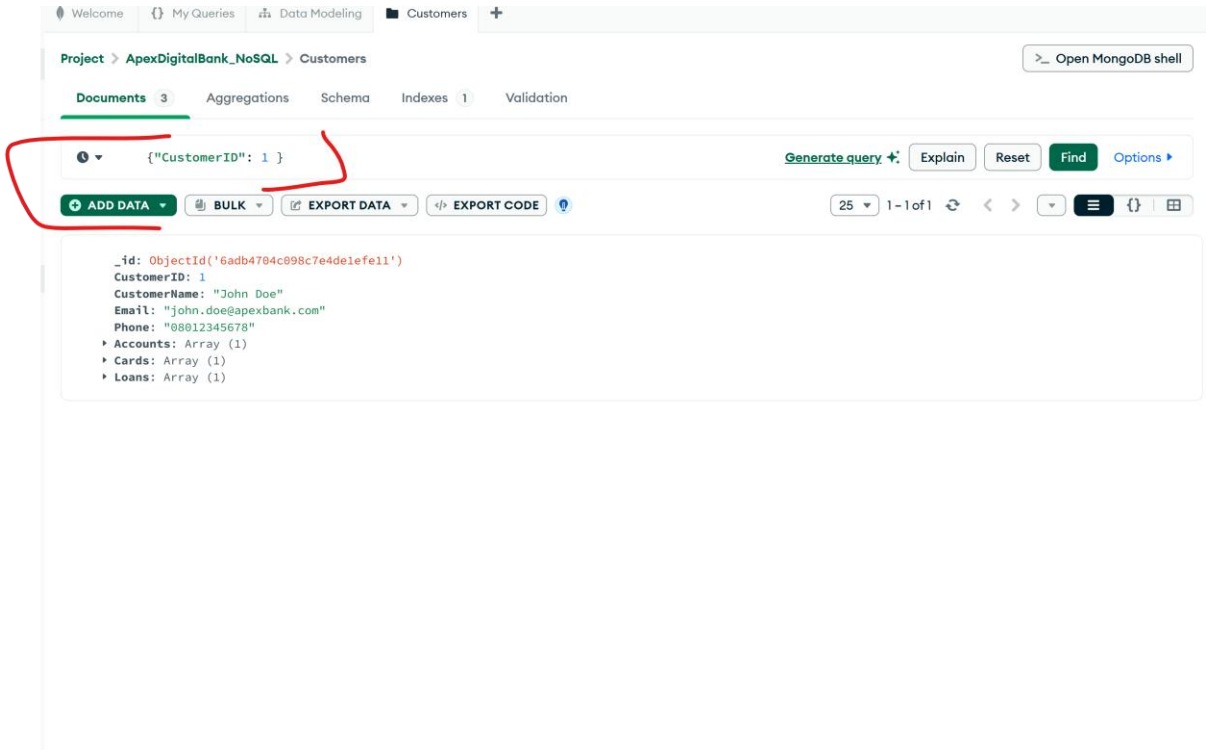
Insert of a few customer documents



Preview Of Data populated



Query Execution Results – Filter for a particular customer.



Customer Document Example

```
{
 "CustomerID": 3,
 "CustomerName": "Michael Johnson",
 "Email": "michael.johnson@apexbank.com",
 "Phone": "08011223344",
 "Accounts": [
  {
   "AccountID": 103,
   "AccountNumber": "1000000003",
   "AccountType": "Savings",
   "Balance": 3200000,
   "Currency": "NGN"
  }
 ]
}
```

Suitability

MongoDB is suitable for Apex Digital Bank because customer information, accounts, cards, and loans can be stored as a single document, reducing complex joins and improving retrieval speed.

This provides schema flexibility, supports rapid development and improves scalability for modern digital banking applications.

2. Key-Value Model

The key-value model stores information as a unique key and corresponding value.

Example

```
{
"Customer_101": {
"Balance": 2500000
 }
}
```

Suitability

The key-value model is suitable for:

- Session management
- User authentication tokens
- Account balance caching
- Fast retrieval operations

Its simplicity enables high-speed access to frequently used banking data.

3. Column-Family Model

The column-family model stores related data together in column groups.

Example

CustomerID: 101

PersonalInfo:

Name

Email

Phone

Accounts:

AccountNumber

Balance

Loans:

LoanAmount

LoanStatus

Suitability

The column-family model is suitable for:

- Large-scale banking datasets
- Analytical processing
- Transaction history storage
- High-performance reporting systems

This model supports horizontal scalability and efficient handling of large volumes of financial records.

4. Graph Model

The graph model represents relationships between entities using nodes and edges.

Example

(Customer)

|

OWNS

|

(Account)

|

PERFORMS

|

(Transaction)

Suitability

The graph model is suitable for:

Fraud detection and relationship analysis because they represent connections between customers, accounts and transactions.

Banks often use graph databases to discover suspicious transaction patterns between accounts.

Comparative Analysis

Model	Strength	Banking Use Case
Document	Flexible schema	Customer profiles and accounts
Key-Value	Very fast lookups	Session caching
Column-Family	Massive scalability	Transaction analytics
Graph	Relationship analysis	Fraud detection

Conclusion

This portfolio has demonstrated the implementation of NoSQL and advanced data models using the Apex Digital Bank case study. MongoDB was used to implement a document-oriented database, while key-value, column-family, and graph models were also evaluated. The analysis showed that different NoSQL models provide unique benefits depending on the banking application requirements, including flexibility, scalability, performance, and relationship analysis.